Network Flow Optimizer

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Sangu Nikhil – 2520030246
VNV Sai Chandu – 2520030522
CH. Pavan Kumar – 2520030312

Team Number: 8

Section: 1

Under the guidance of

Dr. CH. Anuradha

Department of CSE

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled **Network Flow Optimizer** is a record of Bonafide work of Sangu Nikhil-2520030246, VNV Sai Chandu-2520030522, CH. Pavan Kumar-2520030312**.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artificial Intelligence & Data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Sangu Nikhil – 2520030246

VNV Sai Chandu – 2520030522

CH. Pavan Kumar – 2520030312

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project-based report entitled **Network Flow Optimizer i**s a bonafide work done and submitted by Sangu Nikhil-2520030246, VNV Sai Chandu-2520030522, CH. Pavan Kumar-2520030312**.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who have assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. CH. Anuradha, Faculty, CSE which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Sangu Nikhil – 2520030246
VNV Sai Chandu – 2520030522
CH. Pavan Kumar – 2520030312

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	12
8.	Conclusion & Future Scope	12
9.	References	13

1.ABSTRACT

Network Flow Optimizer is a Java application that solves the maximum-flow problem in a directed graph. Each edge has a limited capacity, and the objective is to find the greatest feasible flow from a source to a sink. The application models the network with graph, vertex and edge data structures and implements Dinic's Algorithm and Edmonds–Karp Algorithm. The sample network contains six vertices, numbered 0 through 5, and ten directed capacity edges. Vertex 0 is the source and vertex 5 is the sink. A Java Swing graphical user interface displays the graph and lets users run either algorithm or compare their results and execution times. For the supplied sample graph, both algorithms should calculate a maximum flow of 23 units. The project demonstrates graph representation, residual networks, algorithm design and performance comparison. Execution times can vary according to system load and JVM state.

Team Name:
Network Flow Optimizer

Team Members:

1. Sangu Nikhil – 2520030246
2. V N V Sai Chandu – 2520030522
3. CH. Pavan Kumar – 2520030312

2. INTRODUCTION

Network Flow Optimizer focuses on the maximum-flow problem, a fundamental graph optimization problem. Computer networks, transportation systems and water distribution networks can be represented as directed graphs with capacity-limited connections. The goal is to determine the maximum amount of flow that can travel from a source vertex to a sink vertex without exceeding any edge capacity. This project uses Java data structures to represent the graph, implements Dinic's Algorithm and Edmonds–Karp Algorithm, and provides a Swing interface for running and comparing them. The report describes the related concepts, problem statement, objectives, requirements, implementation, results, conclusion and future improvements.

The introduction is meant to engage the reader, providing enough context to appreciate the project's purpose and importance while also leading smoothly into the project details. It usually ends with a brief outline of what the rest of the document will cover.

3. LITERATURE SURVEY

Maximum-flow algorithms are established techniques in graph theory and combinatorial optimization. The Ford–Fulkerson method uses augmenting paths to increase flow. Edmonds–Karp selects augmenting paths using breadth-first search (BFS), giving a polynomial-time algorithm. Dinic's Algorithm builds a level graph with BFS and sends blocking flow using depth-first search (DFS). Residual networks and reverse edges enable algorithms to revise previous flow decisions. This project applies these concepts in Java and provides a graphical interface to compare the calculated results. Foundational references are listed in the References section.

4. Problem Statement and Objectives

Determine the maximum feasible flow from a source vertex to a sink vertex in a directed graph with capacity-limited edges.

Each connection in a network has a limited capacity. The challenge is to find the greatest total flow from the source to the sink while respecting every edge capacity.

Objectives:

Identify the Project's Purpose:

Objectives: represent a capacitated directed graph; calculate maximum flow using Dinic's and Edmonds–Karp algorithms; display results through a Java Swing GUI; and compare flow values and execution times.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Component - Minimum Requirement

- Processor - Intel Core i3 or equivalent
- RAM - 4 GB
- Hard Disk Space - 500 MB of free space
- Operating System - macOS, Windows, or Linux
- Display - Standard monitor with 1366 × 768 resolution
- Input Devices - Keyboard and mouse

Software Requirements:

- Operating System - macOS, Windows 10/11, or Linux

- Programming Language – Java
- JDK - JDK 17 or later recommended
- IDE / Code Editor - Visual Studio Code, IntelliJ IDEA, or Eclipse
- GUI Framework - Java Swing
- Compiler - Java compiler (javac)
- Runtime Environment - Java Virtual Machine (JVM)
- Version Control - Git and GitHub (optional)

6. IMPLEMENTATION

Module 1 — Graph representation

Graph.java stores vertices and manages edges. Example: Graph graph = new Graph(6); graph.addEdge(0, 1, 16);

Module 2 — Dinic's Algorithm

Dinic.java uses BFS to build a level graph and DFS to send blocking flow through eligible residual edges.

Module 3 — Edmonds–Karp Algorithm

EdmondsKarp.java uses BFS to find an augmenting path, calculates its bottleneck capacity and updates forward and reverse edges.

Module 4 — Graphical interface

MainFrame.java, GraphPanel.java and ControlPanel.java display the network and provide controls to run and compare algorithms.

The implementation is organized into graph data structures, maximum-flow algorithms, GUI components and utility classes. Representative modules are described below.

7. RESULTS & DISCUSSION

The sample network contains six vertices and ten directed edges. With vertex 0 as source and vertex 5 as sink, both algorithms should return a maximum flow of 23 units. The Compare control reports the flow values and measured execution times. Timing depends on the computer and JVM state. Insert screenshots of the running application and results in this section.

8. CONCLUSION & FUTURE SCOPE

Network Flow Optimizer demonstrates how a directed graph with capacity constraints can be processed to calculate maximum flow. It implements Dinic's Algorithm and Edmonds–Karp Algorithm and provides a Swing interface for running and comparing them. On the supplied sample graph, both algorithms should return 23 units. The project connects graph theory, residual networks, algorithm implementation and basic performance evaluation.

Future enhancements include allowing users to edit graphs, add vertices and edges, and configure source and sink. The interface could display actual flow values on edges and implement a functional Reset control. Performance testing could be improved by repeating runs across different graph sizes and reporting average timings. Automated tests and validation for invalid capacities and vertex IDs would also improve reliability.

8. REFERENCES

1. Ford, L. R. Jr. and Fulkerson, D. R., Flows in Networks, Princeton University Press, 1962.

2. Edmonds, J. and Karp, R. M., “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,” Journal of the ACM, 1972.

3. Dinitz, Y., “Algorithm for Solution of a Problem of Maximum Flow in Networks with Power Estimation,” Soviet Mathematics Doklady, 1970.

4. CP-Algorithms, “Edmonds–Karp Algorithm,” https://cp-algorithms.com/graph/edmonds_karp.html

5. CP-Algorithms, “Dinic’s Algorithm,” https://cp-algorithms.com/graph/dinic.html

6. Princeton University, Algorithms — Maximum Flow and Minimum Cut, https://algs4.cs.princeton.edu/64maxflow/

7. Oracle, Java Documentation, https://docs.oracle.com/en/java/

8. Oracle, Java Swing Tutorial, https://docs.oracle.com/javase/tutorial/uiswing/

9. Network Flow Optimizer project source code: graph, algorithm and GUI modules.

10. Data Structures and Algorithms–3 (25CS2103E), course materials.